

Lebanese American University



**School of
Engineering**

COE 324 – Microprocessors Lab Section 13

Mr. Houssein Kanso

Final Project - “Design and Implementation of a 3-Stage Elevator
System Using HCS12 Microcontroller”

Nour Awada 202300006

Jana Monzer 202301360

Ranim Reda 202302524

5/12/2025

Table of Contents

Table of Contents.....	2
Table of Figures.....	3
Introduction	4
Algorithm and Flow Chart	5
Circuit Diagram	Error! Bookmark not defined.
Units Initializing Code	Error! Bookmark not defined.
General Control Variables.....	8
LCD Variables	8
LCD Initialization	8
PWM Initialization.....	8
ATD INIT	8
GPIO Configuration	8
MCCNT Initialization	8
Helping Subroutines	13
LCD Subroutines.....	13
MCCNT Subroutine.....	Error! Bookmark not defined.
LED Subroutines	13
OpenDoorMode	Error! Bookmark not defined.
OverwMode.....	Error! Bookmark not defined.
DoorOverWeightBuzzerOn	Error! Bookmark not defined.
PWMINIT	Error! Bookmark not defined.
PWMOFF	Error! Bookmark not defined.
Potentiometer	Error! Bookmark not defined.
Delay	14
Main Loop	15
Section 1: Overweight check and protection.....	16
Section 2: Requests check and direction decision	Error! Bookmark not defined.
Section 3: Arrival handling at the current floor	Error! Bookmark not defined.
Section 4: Movement Routines	Error! Bookmark not defined.
Evaluation, Analyzing and Testing.....	25
Conclusion.....	26

Table of Figures

No table of figures entries found.

Introduction

This report presents the design and implementation of a three-stage elevator system built on the HCS12 (MC9S12DT256) microcontroller. The project aims to demonstrate advanced embedded-systems techniques, including interrupt-driven input handling, real-time state management, digital/analog interfacing, motion sequencing, and user-interface signaling.

The system supports:

- Internal and external floor selection
- Upward and downward motion sequencing
- Priority-based multi-request scheduling
- LCD messaging
- LED-based direction indication
- Audible arrival alerts
- Overweight detection using an analog potentiometer

The final product behaves as a realistic miniature elevator controller, with smooth transitions, accurate request handling, and full user feedback.

System Objectives:

The primary objectives are:

1. Implement a three-floor elevator controller with correct vertical navigation.
2. Use the MCCNT underflow interrupt to periodically read all push buttons.
3. Maintain correct request queuing and direction prioritization based on current floor.
4. Drive LCD messages, direction LEDs, and arrival buzzer in real time.
5. Implement a weight-sensing safety mechanism using the ATD converter.
6. Produce a modular, optimized assembly program according to COE324 standards.

1. Logic and Flow Chart

2. Hardware Architecture

2.1 Microcontroller Connections

2.2 Inputs

Internal Push Buttons (Cabin)

Button	Floor	Pin
PB3	Floor 0	PA2
PB4	Floor 1	PA3
PB5	Floor 2	PA4

External Push Buttons (On Floors)

Button	Floor	Pin
PB6	Floor 0	PA5
PB7	Floor 1	PA6
PB8	Floor 2	PA7

2.3 Outputs

LED Indicators (PORT T)

LED	Function	Pin
Green	Moving Up	PT7
Yellow	Moving Down	PT6
Red	Door Opening / Arrival / Overweight	PT5

Buzzer

Driven through **PWM Channel 4**, allowing tonal control.

LCD Module

- Interfaced using **SPI (Port M)**
- Operates in **4-bit mode**, following HD44780 timing constraints
- Displays motion status and floor number

Potentiometer (Weight Sensor)

- Connected to **ATD0 Channel 5**
- Converts analog weight equivalent into a 10-bit digital reading
- Overweight threshold defined at 0x66

3. Software Architecture

3.1 Initialization Layer

General Control Variables

These variables store the elevator's real-time operational state, including the current floor, pending internal and external requests, motion direction, and counters used for blink timing and display sequencing. They form the core state Machine that governs all decision-making during elevator movement and arrival handling.

LCD Variables

String buffers such as *MOVING*, *UP*, and *DOWN*, along with their associated size counters, support controlled character-by-character printing to the LCD. These variables enable dynamic messaging through the SPI-driven 4-bit LCD interface and maintain clean formatting during motion and arrival events.

LCD Initialization

The LCD initialization sequence configures the display controller into 4-bit mode, selects the character font, enables the display, and sets cursor behavior. Through a series of timed SENDINST commands, the LCD is brought into a stable operating state that supports both instruction and data transfers for real-time feedback.

PWM Initialization

PWM channel 4 is configured to drive the buzzer with a controllable tone using programmable period and duty-cycle parameters. Initialization sets the clock source, prescaler, polarity, and counter, ensuring consistent acoustic signaling for arrival and overweight conditions.

ATD INIT

The analog-to-digital converter (ATD0) is configured to sample the potentiometer connected to channel 5, providing a 10-bit digital weight reading. The control registers enable the module, select conversion mode, configure resolution and timing, and initiate continuous conversions for real-time overload detection.

GPIO Configuration

PORTA is configured as an input port with internal pull-ups to read push-button states reliably, compensating for mechanical switch behavior. PORTT is configured as an output port to drive the three LED indicators corresponding to upward motion, downward motion, and arrival/overweight status.

MCCNT Initialization

The modulo counter subsystem is initialized to generate periodic underflow interrupts, enabling precise, non-blocking sampling of push-button inputs. Configuration of MCCTL and the counter preload ensures deterministic interrupt timing, allowing the system to update request states independently of the main execution flow.

3.2 Main Control Loop

Weight evaluation

At the beginning of every main-loop iteration, the system evaluates the elevator's load using the ATD conversion result stored in ATD0DR0L. The main loop blocks until a conversion is complete, indicated by the CCF flag in ATD0STAT0. Once the value is read, it is compared against the predefined overweight threshold (CMPA #566). This design ensures that weight evaluation is performed continuously and deterministically, allowing the elevator to react immediately to excessive load conditions. The decision is fully embedded in the control flow, meaning the elevator never proceeds to motion-related tasks unless the weight is confirmed safe.

Normal vs. Overweight Branching

After weight evaluation, the system enters a clear two-path decision: **overweight** or **normal**. If the ATD reading exceeds the threshold, the program branches directly to the overweight routine, where the buzzer is activated, the red LED blinks, and the elevator remains stationary. No motion routines can execute in this state, effectively locking the system in a safety mode. Conversely, if the weight is within limits, the program ensures the buzzer is turned off and transitions to the normal label. This branching acts as a global gatekeeper, guaranteeing that all elevator movement occurs only under safe operating conditions.

Request processing

In the normal state, the system evaluates pending service requests stored in the global requests bitmask. This mask is maintained exclusively by the MCCNT interrupt routine, ensuring up-to-date button information without blocking the main execution flow. The main loop reads this mask and, if it is zero, returns to idle operation. If requests are present, the program compares the current floor value with the active request bits. Using structured conditions within `at_floor0`, `at_floor1`, and `at_floor2`, the system efficiently determines whether the request corresponds to the same floor or requires upward or downward motion. This organization ensures deterministic request resolution and correct ordering for multi-floor scenarios.

Floor-to-floor navigation

Once the requested direction is identified, the system transitions into the appropriate movement routine (`FROMxTOy`). Each routine manages all aspects of travel between two floors: setting the direction flag, selecting the correct LED (green for up, yellow for down), displaying the "MOVING" message, blinking the LEDs for a delay proportional to the floor distance, and updating the current floor variable. These routines encapsulate all timing, signaling, and state updates required for a realistic motion sequence. By compartmentalizing each transition, the code maintains clarity and ensures consistent behavior independent of the starting or target floor.

Arrival handling

After completing a travel routine, the system enters the arrival procedure, which handles all post-motion signaling. This includes activating the buzzer through PWM, blinking the red LED, printing the floor number on the LCD using the floor routine, and finally turning off the LEDs and buzzer before returning to the main loop. The arrival routine also clears the serviced floor's request bit through the logic in the done block, preventing repeated processing of the same request. This structured arrival handling ensures that every completed movement ends with unified feedback to the user and proper state housekeeping within the system.

3.3 Modular Routines

Normal

The normal routine governs standard elevator operation when no overweight condition is present. It first ensures all LEDs are off, then checks whether the requests bitmask contains any pending calls. If requests exist, it proceeds to evaluate the current floor and dispatches control to the appropriate directional decision block (at_floor0, at_floor1, or at_floor2). This routine forms the main execution path for regular elevator motion and request servicing.

Moving Message

The moving_message routine provides dynamic LCD feedback during elevator travel. Upon entering this routine, the LCD is first cleared using a standard command instruction, after which the string "MOVING" is printed character-by-character using the SPI-driven SENDDATA subroutine, paced by controlled delays to ensure timing stability. The routine then checks the direction flag, which is set prior to movement in the transition routines (FROMxTOy). Based on this flag, it automatically branches to either up_message or down_message. These subroutines position the cursor on the second LCD line and print "UP" or "DOWN," providing real-time directional context to the user. The modular structure of this routine ensures that all motion-related display output remains synchronized with actual elevator behavior.

Arrival

The arrival routine centralizes all feedback and signaling associated with reaching a requested floor. It begins by initializing a blink counter and activating the buzzer via PWM4 to generate an audible arrival alert. The red LED is then enabled to signal that the door is opening, followed by a controlled blink sequence using the blink routine. The elevator's current floor is displayed through the floor subroutine, which writes "F" followed by the ASCII value of the current floor to the LCD. Once signaling is complete, the LEDs are turned off and the buzzer is deactivated to conclude the arrival event. This modular approach ensures consistency in how arrival notifications are presented across all floor transitions.

Blink

The blink routine implements timed LED blinking for motion and arrival signaling. It operates by repeatedly outputting the LED pattern stored in register B to PTT, holding the output through DELAY2, clearing the LEDs, and delaying again to form a full blink cycle. The routine decrements the global counter variable each cycle, determining how many blinks occur before exiting. Because blink is invoked by motion routines, overload handling, and arrival processing, it provides a unified timing mechanism that maintains consistent visual output regardless of system state. By abstracting blink behavior into its

own routine, the code eliminates redundancy and ensures reliable, hardware-consistent LED modulation.

Green, Yellow, Red

The LED color routines—green, yellow, and red—provide straightforward, isolated control over the elevator’s direction and status indicators. Each routine writes a specific bit pattern to PTT corresponding to PT7 (green), PT6 (yellow), or PT5 (red), enabling only the intended LED while clearing the others. They also load this same pattern into register B so that subsequent routines, such as blink, can reuse the LED state without recomputing it. These color routines are used throughout the codebase to reflect motion direction, door status, and overload conditions. Their modularity ensures that LED behavior remains consistent and easily maintainable, with all direction/status signaling originating from clearly defined, reusable subroutines.

3.4 Helping Subroutines

LCD Subroutines

The LCD subsystem is implemented through the modular routines SENDINST, SENDDATA, and SENDSPI. These subroutines collectively establish a full 4-bit SPI-driven communication interface with the LCD controller. SENDINST handles command instructions. SENDDATA operates similarly but transmits ASCII characters for display. Both routines rely on SENDSPI, which performs the actual SPI transfer by polling the SPTEF flag and writing the data to SPI0DR. The modularity of these routines ensures stable LCD timing, proper nibble sequencing, and clean abstraction from the main logic, allowing higher-level routines such as moving_message and floor to display dynamic content without managing low-level signaling.

LED Subroutines

LED control is encapsulated in the green, yellow, red, and off routines. Each routine writes a specific bit pattern to the PTT register to activate only the LED corresponding to the elevator's state: green for upward motion, yellow for downward motion, red for arrivals or warnings. Additionally, they load the same pattern into register B, ensuring compatibility with the blink routine, which depends on B for repeated output. This modular structure isolates all LED state assignments, eliminating ambiguity and ensuring consistent visual feedback throughout elevator operation.

Buzzer ON

The buzzer_on subroutine activates the audible alert by configuring PWM channel 4 with a fixed period (PWMPER4 = 250) and a 50% duty cycle (PWMDTY4 = 125). These parameters generate a stable, continuous tone used for arrival notifications and overweight alarms. By abstracting PWM register configuration into a single routine, the system ensures consistent acoustic behavior whenever the buzzer is required.

Buzzer Off

The buzzer_off subroutine disables the buzzer by clearing both the PWM period and duty cycle registers. Setting PWMPER4 and PWMDTY4 to zero guarantees immediate silence and prevents any unintended PWM activity. This routine ensures clean transitions out of alarm or arrival states by reliably stopping sound output.

Up message

The up_message routine manages the second-line LCD output when the elevator is moving upward. It positions the cursor at the beginning of line two (SENDINST with 0xC0) and prints the two-character string "UP" sequentially using SENDDATA. This routine is invoked after the general "MOVING" header has been displayed and reinforces the directional context of upward travel.

Down message

The down_message routine provides the corresponding downward travel indication. Like its upward counterpart, it writes to the second LCD line and prints "DOWN" one character

at a time. It also ensures that the direction variable is cleared, marking the motion as downward internally. This messaging ensures user clarity and synchronizes the LCD display with the direction indicated by the LEDs.

Floor

The floor routine displays the elevator's current floor at arrival. It begins by clearing the LCD, printing the character "F," and then converting the current floor value into its ASCII equivalent by adding 48 before sending it to the display. This standardized floor-labeling format ensures a clear, unambiguous visual confirmation of the elevator's final position after each movement cycle.

Done

The done block is responsible for clearing the request bit associated with the floor that has just been serviced. It uses the FLOOR_MASKS lookup table to retrieve the bitmask for the current floor, inverts it, and applies it to the requests byte to remove the completed request. This ensures that serviced floors are not reprocessed and keeps the request queue accurate as new button inputs arrive.

Same floor

The same_floor label captures scenarios in which the elevator is already located at a requested floor. It directly invokes the arrival routine, bypassing any motion sequences. This ensures that the system responds appropriately to same-floor calls—such as door opening requests from inside the elevator—without unnecessarily engaging the motion or blink subsystems.

Delay Subroutines

The program uses three delay mechanisms—DELAY, DELAY2, and DELAYarrived—to create deterministic timing intervals across LCD writing, LED blinking, and arrival animations. DELAY provides a short timing gap for LCD nibble writes, while DELAY2 generates longer intervals for LED blink cycles. DELAYarrived uses a nested loop combined with user-adjustable DIP switch values to control arrival dwell time. These subroutines ensure clean timing behavior without blocking critical interrupt-driven request updates.

4. Interrupt Handling and Request Management

The elevator system relies extensively on an interrupt-driven architecture to ensure real-time responsiveness to all floor requests. Push-button polling is not performed in the main loop; instead, it is entirely delegated to the **MCCNT underflow interrupt**, which provides deterministic timing and avoids blocking delays. This design ensures that user inputs are captured promptly, independent of the ongoing motion, LCD operations, or buzzer activity.

When the modulo counter underflows, the interrupt service routine (ISR) is triggered. The routine first acknowledges the interrupt by clearing the MCZF flag and reloads the counter value to maintain consistent periodic execution. It then reads the raw button states from **PORTA**, inverts them to compensate for pull-ups, and shifts the bits to align internal and external push buttons into their designated positions. The ISR separately extracts the three internal cabin buttons and the three external floor-call buttons using masking operations. Through additional right shifts, the internal request bits are normalized and stored in the `internal` variable. These values are then combined with the external requests and merged into the global `requests` bitmask, which serves as the central queue of pending service calls.

This interrupt-based request management strategy provides several advantages. It guarantees that button inputs are sampled at regular intervals, unaffected by motion delays, LCD printing, or LED blinking. It also ensures that no request is missed, even when multiple buttons are pressed in rapid succession or simultaneously. The `requests` bitmask maintained by the ISR is then used by the main loop to determine whether the elevator should ascend, descend, or perform same-floor arrival processing. In addition, because request bits persist until explicitly cleared in the `done` routine, the system behaves as a proper queued controller rather than reacting only to instantaneous button presses.

Overall, the interrupt subsystem forms the backbone of the control logic. It decouples input acquisition from motion handling, guarantees fairness between internal and external calls, and maintains a clean, consistent request structure that drives all scheduling and navigation decisions in the elevator system.

5. Motion Scheduling Logic

The elevator's motion control is governed by a structured decision-making process that interprets request patterns, determines the appropriate direction of travel, and executes movement routines with synchronized signaling. All motion decisions originate from the requests bitmask and the current floor stored in `current`. The elevator operates deterministically, ensuring that every requested floor is served in a logically ordered sequence without ambiguity or conflicting behaviors. The following subsections describe how the system manages individual requests, multiple simultaneous requests, and the full implementation of floor-to-floor travel.

5.1 Single Requests

When a single request is active, the control logic resolves it by comparing the requested floor's bit with the current floor value. This behavior is implemented through the dispatch blocks at `at_floor0`, `at_floor1`, and `at_floor2`, each of which evaluates the corresponding bit in the requests mask. If the request matches the current floor, the program invokes `same_floor` to trigger the arrival sequence without performing any movement. If the request lies above the current floor, the logic branches to the appropriate upward routine, such as `FROM0TO1` or `FROM1TO2`. Conversely, if the request lies below the current floor, the system branches to downward routines like `FROM1TO0` or `FROM2TO0`. This approach ensures a direct, unambiguous mapping between a single request and the correct motion path.

5.2 Multiple Simultaneous Requests

When multiple floor requests occur simultaneously, the elevator resolves them in ordered sequences that reflect realistic scheduling principles. If all active requests lie above the current floor, the system serves them in **ascending order**, transitioning sequentially from lower to higher floors. If all requests lie below the current floor, they are served in **descending order**, from higher floors downward. This behavior naturally emerges from the conditional branching in the floor-specific logic blocks, where the program checks the lowest or highest relevant bit first depending on the elevator's position. The ISR maintains the requests bitmask until the `done` routine clears individual bits after arrival, ensuring that no floor is skipped and that the elevator follows a predictable, queue-like service pattern during multi-request scenarios.

5.3 Motion Implementation

Motion execution is encapsulated in dedicated routines (`FROM0TO1`, `FROM0TO2`, `FROM1TO2`, `FROM1TO0`, `FROM2TO1`, and `FROM2TO0`) that handle all aspects of travel between floors. Each routine performs a series of coordinated operations: it sets the direction variable, activates the appropriate LED (green for upward, yellow for downward), displays the "MOVING" message followed by the specific direction, and triggers the blink routine to simulate travel duration using controlled delay cycles. The travel delay is proportional to the distance between floors, with longer movements assigning larger blink counters. After completing the motion cycle, the routine updates the current floor and invokes `DELAYarrived` to simulate door opening before transitioning

into the arrival routine. By encapsulating all movement-related behavior within these modular routines, the system ensures consistent timing, clear visual/auditory signaling, and clean transitions from motion to arrival.

6. Safety Overload Handling

The overweight detection subsystem is a critical safety component of the elevator controller, designed to prevent motion when the cabin load exceeds a defined threshold. This feature is implemented using the HCS12's ATD (Analog-to-Digital Converter) module, which continuously samples the potentiometer connected to channel 5. The potentiometer serves as the electronic weight sensor, providing a 10-bit digital measure of the applied load. Initialization of the ATD converter (`ATD0CTL2`, `ATD0CTL3`, `ATD0CTL4`, and `ATD0CTL5`) configures the module for continuous, right-justified conversions at full resolution, ensuring reliable and noise-free analog sampling throughout system operation.

During each iteration of the main control loop, the program waits for the ATD conversion complete flag and reads the latest sample from `ATD0DR0L`. This value is then compared against a predefined safety threshold (`CMPSA #566`), which represents the maximum allowable load. If the measured weight falls within normal operating limits, the system proceeds to the standard elevator logic. However, if the reading exceeds the threshold, the program immediately branches to the dedicated `overweight` routine. This safety handler activates the buzzer using PWM (`buzzer_on`), triggers the red LED for visual alerting (`red`), and performs a controlled blink cycle to signal a persistent hazard condition.

While in the overweight state, elevator motion is completely locked out: none of the movement routines (`FROMxTOy`) can execute, preventing unsafe operation under excessive load. The system remains in this mode until subsequent ATD readings fall below the threshold, at which point control returns automatically to the main loop and normal elevator behavior resumes. By isolating overweight handling into a dedicated subsystem and tying it directly into the main loop's state-transition logic, the implementation ensures that safety always overrides user requests and operational commands. This robust design models real-world elevator safety protocols by halting movement, alerting users, and restoring service only after safe load conditions have been re-established.

7. User Interface Components

7.1 LED System

The LED subsystem utilizes three output pins on Port T—PT7 (green), PT6 (yellow), and PT5 (red)—to signal elevator motion and operational status. Each LED is controlled through modular routines (green, yellow, red, and off), which write predefined bit patterns to the PTT register. During upward motion, the program invokes the green routine, illuminating PT7; downward motion activates the yellow routine via PT6; and arrival or safety conditions trigger the red LED through the red routine on PT5. The blink subroutine further enhances visual signaling by toggling the active LED pattern on and off for a duration determined by the counter variable. This creates a consistent blinking effect for movement animations and alert conditions. By abstracting LED control into discrete routines, the system maintains clean separation between motion logic and output signaling while ensuring that visual indicators remain synchronized with the underlying operational state.

7.2 Buzzer

Auditory feedback is delivered through the buzzer connected to PWM channel 4, controlled by the `buzzer_on` and `buzzer_off` routines. The buzzer is enabled by configuring a fixed period ($PWMPER4 = 250$) and a 50% duty cycle ($PWMDTY4 = 125$), producing a clear and steady tone. This signal is used to reinforce important states: short tones during arrival and continuous sound during overweight conditions. The PWM-based design ensures stable frequency generation free of timing jitter and allows sound control without blocking or delaying the main program flow. By centralizing tone control within the PWM subsystem, the implementation aligns with embedded systems best practices, ensuring reliable auditory alerts that complement visual indicators.

7.3 LCD Messaging

The LCD messaging system provides text-based feedback to clearly communicate current motion, direction, and floor status. Communication with the LCD is implemented via SPI-driven 4-bit transfers using the `SENDINST`, `SENDDATA`, and `SENDSPI` routines. High-level messaging routines such as `moving_message`, `up_message`, `down_message`, and `floor` ensure structured, readable output aligned with system state transitions. During travel, the LCD displays “MOVING” followed by either “UP” or “DOWN” depending on the direction flag, reinforcing direction signaling already reflected by the LEDs. Upon arrival, the floor routine prints the current floor value (“F0,” “F1,” or “F2”) to confirm completion of the movement sequence. Carefully placed delays ensure accurate nibble timing in compliance with the LCD controller’s protocol. Overall, the LCD subsystem provides immediate, user-friendly textual insight into elevator operation, enhancing clarity and operational transparency.

8.Full Program Flow

The operation of the elevator controller follows a structured execution flow built around a combination of initialization routines, a persistent main loop, and periodic interrupt service routines. This architecture ensures deterministic response, continuous monitoring of user requests, and safe, coordinated motion control. The following subsections describe the complete flow from system startup to real-time operation.

Power-Up

Upon reset, the program begins at the Entry label, where essential system components are initialized to guarantee stable and predictable behavior. The stack pointer is configured, interrupts are enabled, and peripheral modules—including SPI for LCD communication, PWM for buzzer control, ATD for weight sensing, and GPIO directions—are fully initialized. The LCD undergoes a complete configuration sequence to enter 4-bit mode and prepare for text output. LEDs are cleared through an initial write to PTT, and the modulo counter (MCCNT) is configured to generate periodic underflow interrupts. By the end of this phase, all hardware interfaces are ready, and the system transitions into the main control loop with a known, stable baseline state.

Normal Operation

Normal operation is governed by the main loop, which continuously evaluates the elevator's environment and determines the appropriate actions. At each iteration, the system first checks the ATD conversion status and reads the weight sensor value. If the threshold is exceeded, execution branches to the overweight routine, halting movement and enabling both visual and audible warnings. When weight conditions are safe, the system proceeds to evaluate pending service requests stored within the requests bitmask. If no requests exist, the elevator idles until new inputs arrive. If requests are present, the system determines whether the requested floor is above, below, or equal to the current floor and routes execution to the proper transition routines (FROMxTOy), arrival logic (same_floor and arrival), and post-travel cleanup (done). Throughout this loop, direction indicators, LCD updates, and blink cycles are activated as needed, ensuring the elevator behaves predictably and responsively.

Interrupt-Driven Updates

Real-time responsiveness is maintained through interrupt-driven request acquisition. The MCCNT underflow interrupt triggers the ISR routine periodically, independent of the main loop's workload. Within the ISR, raw button states from PORTA are read, inverted, masked, and shifted to align internal and external inputs into their proper positions. These values are merged into the internal buffer and then OR-combined with the global requests bitmask, ensuring that all pending floor calls—whether internal or external—are captured reliably. The ISR's adds non-blocking structure guarantees that button presses

are never missed, even during long delays associated with movement animations, LCD writes, or buzzer activity. By separating input acquisition from operational logic, the interrupt subsystem maintains consistent system responsiveness and ensures that motion decisions in the main loop always reflect the most recent user inputs.

Floor Navigation Logic

The floor navigation subsystem determines how the elevator evaluates its current position, interprets pending requests, and transitions between floors in an orderly and deterministic manner. This subsystem is built around two complementary components: the **at_floorX** decision blocks, which determine the next required action based on current floor state and the request bitmask, and the **FROMXTOY** motion routines, which perform the actual movement between floors. Together, they form the core of the elevator's scheduling and navigation engine.

At-Floor Decision Blocks (at_floor0, at_floor1, at_floor2)

Each floor has a dedicated decision block that evaluates the requests bitmask in relation to the elevator's current floor. These blocks serve as localized state handlers, ensuring that movement decisions are made according to the elevator's present position.

- **at_floor0** checks whether floor 0 has been requested. If the floor 0 bit is set, the system immediately performs same-floor arrival handling. If not, it evaluates requests for floors 1 and 2 in ascending order, branching into FROM0TO1 or FROM0TO2 depending on which bit is active.
- **at_floor1** handles the central floor, responding first to a same-floor request and then checking whether requests exist above (floor 2) or below (floor 0). This ensures proper directional decision-making in multi-request scenarios, allowing floor 1 to act as a midpoint dispatcher.
- **at_floor2** mirrors the logic of floor 0 but in descending order. It first checks for a same-floor request, then scans for lower-floor requests, routing execution into FROM2TO1 or FROM2TO0 accordingly.

Through these conditional structures, the elevator interprets the pending requests in a way that respects natural elevator behavior—serving higher floors when above the current position and lower floors when below—while ensuring deterministic motion during multi-request handling.

Floor Transition Routines (FROMXTOY)

The actual movement between floors is carried out by the structured motion routines (FROM0TO1, FROM0TO2, FROM1TO2, FROM1TO0, FROM2TO1, and FROM2TO0). Each motion routine encapsulates:

1. **Direction Assignment**

The direction flag is set or cleared depending on whether the elevator is moving upward or downward.

2. User Feedback Activation

The `moving_message` subroutine displays “MOVING” and the appropriate direction (“UP” or “DOWN”) on the LCD. The direction-specific LED (green for upward, yellow for downward) is enabled to provide clear external signaling.

3. Simulated Travel Timing

The number of blink cycles executed by the blink routine reflects the travel duration. Shorter distances (one floor) use fewer cycles; longer distances (two floors) use extended cycles by adjusting the counter variable.

4. Floor State Update

Upon completion of the travel delay, the current variable is updated to the new floor index.

5. Arrival Timing and Cleanup

The elevator pauses using the `DELAYarrived` routine to simulate door opening, then transitions to `same_floor` or the arrival routine for red LED signaling, buzzer activation, and floor display.

These routines fully encapsulate the behavior associated with moving from one floor to another, providing clarity, modularity, and consistency across all floor transitions.

Integrated Navigation Behavior

By coupling the **at_floorX** decision blocks with the **FROMXTOY** motion routines, the elevator’s navigation system achieves a clean separation of concerns:

- **at_floorX** routines determine *what to do next* based on button requests and elevator position.
- **FROMXTOY** routines determine *how to execute that movement* safely and with proper user feedback.

This structure ensures that the system handles single requests, multiple simultaneous requests, prioritized direction handling, and same-floor arrival conditions with equal reliability. The modular nature of these routines also makes the navigation logic highly maintainable and extensible, supporting future modifications such as additional floors or enhanced scheduling algorithms.

9. Testing and Validation

Correct Handling of All Button Combinations

The system was rigorously tested to ensure reliable interpretation of all internal and external push-button inputs under the interrupt-driven architecture. Multiple combinations were evaluated, including isolated button presses, simultaneous multi-floor requests, and mixed internal-external inputs. The MCCNT underflow interrupt consistently captured button states without loss or jitter, validating the correctness of the bit-normalization and masking logic implemented in the ISR. The reconstructed bit-alignment ensured that each request correctly updated the corresponding bit in the requests variable, and subsequent processing in the main loop confirmed that the prioritization rules were applied exactly as intended. The system demonstrated stable behavior even under rapid, repeated button activations.

Correct Upward/Downward Motion

Motion routines (FROMxTOy) were examined for every valid floor transition to verify accurate direction handling and state progression. Each routine correctly updated the direction flag, performed the appropriate LED activation, invoked the “MOVING” and directional LCD messages, executed the required number of blink cycles, and ultimately updated the current floor value. The control flow between states remained deterministic across all transitions, and the masking of served requests after arrival ensured that the elevator never re-entered a completed floor state. Testing validated that ascending requests were always processed in ascending order and descending requests in descending order, fully matching the scheduling logic implemented in the decision blocks (at_floor0, at_floor1, at_floor2).

Output Signaling

The output signaling subsystem was validated to ensure perfect synchronization between logical states and physical feedback. Green and yellow LEDs accurately reflected upward and downward motion, respectively, with proper blinking achieved through the blink routine and counter-based timing. The red LED activated reliably during arrival and overweight scenarios. The LCD interface—driven through the SPI-based 4-bit protocol—successfully displayed “MOVING,” “UP,” “DOWN,” and the final floor identifier without flicker or corruption, confirming that the SENDINST and SENDDATA routines met the LCD’s timing requirements. The buzzer, controlled via PWM4, produced consistent tonal output during arrival events, validating duty-cycle configuration and proper activation/deactivation sequencing.

Overweight Behavior

The overweight protection mechanism was evaluated using a range of potentiometer values to simulate varying load conditions. When the ATD conversion result exceeded the predefined threshold, the system immediately halted all elevator movement and entered a controlled alarm state. In this state, the buzzer remained continuously active and the red LED blinked persistently, while the main loop prevented any transition into motion routines. Once the simulated weight returned to an acceptable level, the system disengaged the alarm state, resumed normal LED behavior, and re-enabled request processing. These tests confirmed that the overweight logic correctly integrated ATD readings, threshold comparison, and state isolation, ensuring that safety constraints override all other system functions.

10. Challenges and Resolutions

During implementation and testing, several technical issues were identified and resolved to ensure stable and predictable system behavior.

Incorrect LED Blink During Upward Motion

One significant issue involved the LED direction indicators, where the yellow LED occasionally remained active during upward motion. This was traced to the absence of a proper clearing step before assigning a new LED state and was corrected by explicitly resetting the PTT register prior to enabling the appropriate direction indicator.

LCD Initialization Instability

The LCD module also presented initialization instability due to its strict timing requirements in 4-bit mode; introducing additional delay loops resolved the synchronization problem and produced consistent display output.

Incorrect Button Mapping

The system initially suffered from incorrect button mapping, caused by misaligned bit-shifting operations in the interrupt routine. The normalization logic in the ISR was reconstructed to ensure reliable separation of internal and external requests.write

Arrival Routine Re-Entry

A more subtle issue emerged in the arrival handling flow, where invoking the same_floor routine from within motion routines risked unintended re-entry and register corruption. This was addressed by verifying routine layering and preserving register states, ensuring a clean and deterministic control flow throughout the system.

Conclusion

The development of the three-stage elevator system successfully demonstrates the integration of hardware interfacing, real-time control, and modular software design on the HCS12 microcontroller platform. Through the coordinated use of interrupts, analog-to-digital conversion, PWM signaling, and SPI-driven LCD output, the system achieves reliable floor selection, precise motion control, and intuitive user feedback. The implementation meets all functional requirements specified for the project, including prioritized request scheduling, accurate direction indication, stable display messaging, and robust overweight protection.

Extensive testing confirmed that the elevator responds consistently under a variety of input scenarios, including simultaneous button presses and rapid request sequences. Issues encountered during developments such as LED state conflicts, LCD timing sensitivity, button-mapping discrepancies, and control-flow risks—were systematically diagnosed and resolved, leading to a stable and optimized final product.

Overall, this project provides a practical demonstration of embedded-systems engineering principles, highlighting the importance of timing accuracy, interrupt-driven architecture, and structured program organization. The resulting system is a dependable and extensible platform that can serve as a strong foundation for more advanced elevator control features or broader real-time control applications.